Architectural Blueprint to Facade Renderer: Paired Semantic Layout-to-Image Translation Using a Conditional Generative Adversarial Network

Arpit Gupta, Aryan Tiwari
Department of Computer Science
KIET Deemed to be University
Ghaziabad, Uttar Pradesh, India

***Abstract*—Architectural facade visualization is a useful application of generative image translation because a structural description does not directly communicate the appearance of materials, windows, doors, and other visual details. This project implements a Pix2Pix-style conditional generative adversarial network (cGAN) to translate paired semantic facade annotations into facade photographs. The implementation is developed in PyTorch and uses a U-Net generator and a PatchGAN discriminator. The dataset pipeline matches facade photographs with their corresponding annotation images, prepares paired examples, and partitions the data into training, validation, and test subsets. The generator is trained with adversarial and reconstruction objectives, while validation loss is used to select a model checkpoint. The notebook evaluates generated outputs through visual inspection, mean absolute error (MAE), and structural similarity (SSIM). This paper describes the task, data pipeline, model design, training procedure, evaluation protocol, limitations, and directions for future work. Quantitative results are reported only when supported by the executed notebook; unavailable measurements are marked as not reported. The current experiment uses semantic facade layouts rather than engineering CAD blueprints; therefore, it should be interpreted as semantic layout-to-facade translation, not as a complete CAD rendering system.**

***Index Terms*—conditional generative adversarial network, Pix2Pix, image-to-image translation, U-Net, PatchGAN, facade synthesis, PyTorch**

I. Introduction

Architectural visualization helps communicate how a building may appear before a final design is constructed. Semantic facade layouts represent the arrangement of facade elements such as walls, windows, and doors, but do not contain the complete appearance of a real facade. Image-to-image translation offers a way to learn a mapping from such structured inputs to plausible images using examples of corresponding input and target pairs.

This project investigates paired semantic layout-to-facade translation using a conditional generative adversarial network. Given a semantic facade annotation, the generator predicts a facade image. A discriminator then evaluates whether a facade image paired with the same input layout appears to come from the real dataset or from the generator. A reconstruction loss encourages the generated output to remain similar to its paired target.

The assigned project title is "Architectural Blueprint to Facade Renderer." However, the implemented dataset consists of semantic facade annotations paired with facade photographs, rather than detailed CAD drawings or floor plans. For technical accuracy, the experiment is described throughout this paper as semantic layout-to-facade image translation.

The main contributions of this coursework project are:

- implementation of a paired image-to-image generation pipeline using PyTorch;
- use of a U-Net generator and PatchGAN discriminator in a Pix2Pix-style cGAN;
- training with adversarial and L1 reconstruction objectives, with validation-based checkpoint selection; and
- evaluation using generated-image inspection, MAE, and SSIM.

II. Related Work

Generative adversarial networks (GANs) train a generator and discriminator in competition, allowing the generator to learn the distribution of realistic examples [1]. Conditional GANs extend this setup by conditioning generation on additional information, such as an input image. Isola *et al.* introduced Pix2Pix as a general framework for paired image-to-image translation, combining a conditional adversarial objective with a reconstruction loss [2]. Its U-Net generator preserves spatial information through skip connections, while a PatchGAN discriminator focuses on local image structure and texture. These ideas provide the conceptual basis for the implementation used in this project.

III. Problem Formulation

Let x represent a semantic facade annotation and y its paired real facade photograph. The objective is to learn a generator G such that $G(x)$ resembles the target image y while remaining visually plausible. A discriminator $D(x, y)$ receives the input layout together with either the real target or the generated target and predicts whether the pair is real or generated.

The task is supervised because each training input has a corresponding target image. The desired output is a candidate visualization, not a construction-ready architectural drawing. The model does not infer structural safety, building dimensions, materials compliance, or engineering constraints.

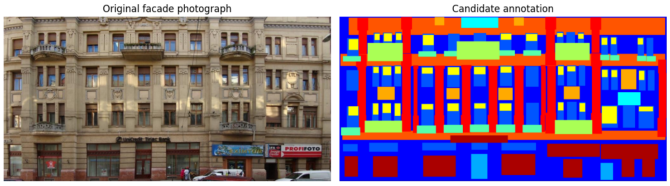


Fig. 1. Example paired data showing a semantic facade annotation and its corresponding facade photograph.

IV. Dataset and Preprocessing

A. Dataset

The project uses the CMP Facade Database and its available extended data. The notebook identifies facade photographs and their corresponding PNG annotation files, then constructs matched pairs. The source dataset is available from the Center for Machine Perception facade database [3]. The annotations encode semantic facade regions and are used as conditioning inputs; photographs serve as target images.

B. Pair Matching and Inspection

The data preparation pipeline searches the downloaded dataset for image files and matches each photograph to an annotation. A visual inspection step displays example pairs before training. This step is important because a mismatch between a photograph and its annotation would provide an incorrect training target and impair the learned mapping.

C. Image Preparation and Splits

The notebook resizes input and target images to 256×256 pixels, converts them to three-channel RGB tensors, and normalizes pixel values to the range $[-1, 1]$. It partitions the paired data into training, validation, and test subsets using an 80% / 10% / 10% split. The training subset updates the model parameters, the validation subset supports checkpoint selection, and the test subset is reserved for final evaluation.

V. Methodology

A. System Overview

The pipeline consists of paired data preparation, generator inference, discriminator evaluation, loss computation, model optimization, checkpoint selection, and final evaluation. During training, the generator receives the semantic layout and produces a candidate facade. The discriminator receives the same layout paired with either the real facade or the generated facade. Both networks are updated through backpropagation according to their respective objectives.

B. U-Net Generator

The generator follows a U-Net-style encoder–decoder design. The encoder progressively extracts higher-level features while reducing spatial resolution. The decoder reconstructs an image from these features. Skip connections pass information from encoder layers to corresponding decoder layers, helping preserve spatial details such as the positions and shapes of facade elements.

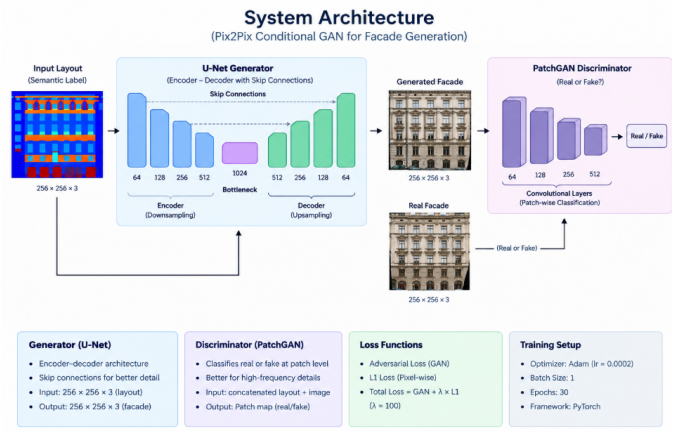


Fig. 2. System architecture of the Pix2Pix-style conditional GAN, including the U-Net generator and PatchGAN discriminator.

C. PatchGAN Discriminator

The discriminator uses a PatchGAN-style design to classify local regions of a conditional image pair as real or generated. It receives the semantic input together with a target facade. During training, real pairs comprise the input annotation and its corresponding photograph, while generated pairs comprise the same annotation and the generator output. Conditioning the discriminator on the input encourages the generated image to correspond to the provided layout rather than merely look like an arbitrary facade.

D. Training Objective

The generator is optimized using an adversarial objective and an L1 reconstruction objective. In the implemented configuration, the combined generator objective is expressed as

$$\mathcal{L}_G = \mathcal{L}_{\text{GAN}} + \lambda_{L1}\mathcal{L}_{L1}, \tag{1}$$

where $\mathcal{L}_{\text{GAN}}$ encourages outputs that the discriminator classifies as real, and $\mathcal{L}_{L1}$ penalizes pixel-wise differences between the generated image and the paired target. The notebook uses $\lambda_{L1} = 100$. The discriminator is trained to distinguish real conditional pairs from generated conditional pairs.

The L1 term is defined as

$$\mathcal{L}_{L1} = \mathbb{E}_{x,y}\left[\|y - G(x)\|_1\right]. \tag{2}$$

The adversarial term and reconstruction term have complementary roles: the former encourages plausible local appearance, while the latter encourages correspondence to the paired target.

E. Optimization and Checkpoint Selection

The notebook uses the Adam optimizer with a learning rate of 0.0002 and $\beta_1 = 0.5$. The configured training duration is 30 epochs. The best generator checkpoint is selected using validation L1 loss rather than assuming that the final epoch is necessarily the best. The actual number of completed epochs and the runtime hardware should be recorded from the executed notebook when reporting the final experiment.

TABLE I
IMPLEMENTATION CONFIGURATION

Component	Configuration
Deep learning framework	PyTorch
Task	Paired image-to-image translation
Generator	U-Net
Discriminator	PatchGAN
Input resolution	256×256 RGB
Data split	80% / 10% / 10%
Optimizer	Adam
Learning rate	0.0002
Adam β_1	0.5
Configured epochs	30
Checkpoint criterion	Validation L1 loss
Evaluation	MAE and SSIM

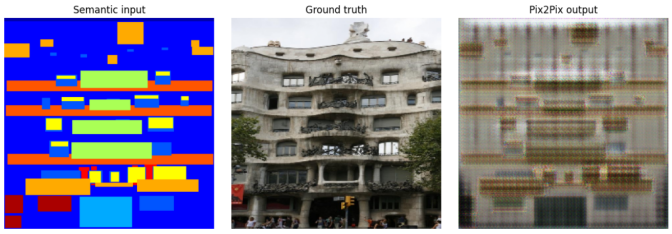


Fig. 3. Qualitative comparison of the input annotation, target facade, and generated prediction. The prediction should be taken from the executed notebook.

VI. IMPLEMENTATION

The project is implemented in Python using PyTorch. The notebook downloads and inspects the dataset, builds the image-pair list, creates the train/validation/test datasets, defines the U-Net generator and PatchGAN discriminator, trains the models, saves checkpoints, plots loss histories, and evaluates the selected generator.

VII. EXPERIMENTAL RESULTS

The notebook evaluates the selected generator on held-out test examples. The qualitative evaluation compares the semantic input, real target facade, and generated prediction. Quantitative evaluation reports mean absolute error (MAE) and structural similarity (SSIM). MAE measures average pixel-level absolute difference, whereas SSIM estimates similarity in local image structure, luminance, and contrast. Neither metric alone fully captures architectural realism or whether an image would be useful in a real design workflow.

A. How to Report the Results

The current report does not state numerical test metrics because verified values were not available in the supplied source. Add measured values only after checking the executed notebook. Describe visible strengths and failure cases based on genuine outputs, such as window alignment, facade texture, and remaining artifacts. Do not claim successful photorealistic generation unless the displayed results support that conclusion.

TABLE II
TEST-SET RESULTS FROM THE EXECUTED NOTEBOOK

Metric	Measured value
Test MAE	Not reported
Test SSIM	Not reported
Completed training epochs	Not verified
Test examples evaluated	Not reported

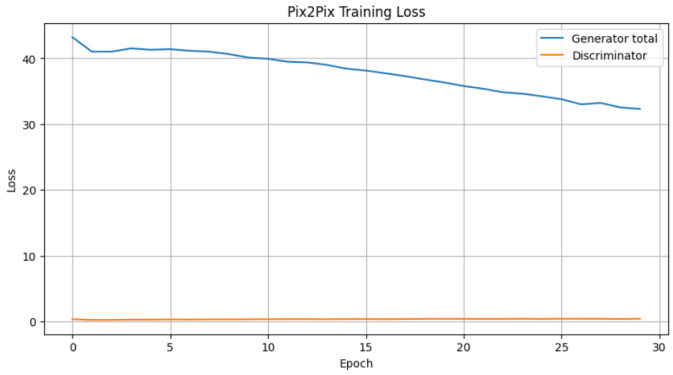


Fig. 4. Training loss history from the executed notebook, when available.

VIII. LIMITATIONS AND FUTURE WORK

The current experiment has several limitations. First, the dataset represents semantic facade annotations paired with facade photographs, not actual CAD blueprints, floor plans, or construction drawings. Second, the quality and diversity of generated images depend on the quantity and correctness of the matched training pairs. Third, adversarial training can be sensitive to optimization and may produce blurry, repetitive, or structurally inconsistent outputs. Fourth, MAE and SSIM are useful complementary metrics but do not fully assess semantic correctness, architectural feasibility, or human-perceived quality. Finally, the system generates conceptual images and should not be treated as a substitute for professional architectural design or engineering validation.

Future work could evaluate additional training checkpoints, expand the paired dataset, compare against a suitable baseline, and include perceptual metrics or human assessment. A true blueprint-to-rendering system would require an appropriate dataset pairing engineering drawings or CAD representations with corresponding rendered images, along with a separate evaluation of geometric and architectural constraints.

IX. CONCLUSION

This project implements a Pix2Pix-style conditional GAN for semantic layout-to-facade image translation. A U-Net generator predicts a facade image from a semantic annotation, while a PatchGAN discriminator provides adversarial feedback conditioned on the same input. The training objective combines adversarial loss with a weighted L1 reconstruction term, and the notebook selects a generator checkpoint using validation L1 loss. The final quality of the system must be

established using actual generated examples and test-set measurements. The current implementation provides a coursework-scale exploration of conditional image generation, while the distinction between semantic layouts and real CAD blueprints defines an important boundary for interpreting its results.

Acknowledgment

The authors acknowledge the CMP Facade Database for providing paired facade imagery and annotations used in this coursework project.

References

[1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems*, vol. 27, 2014.

[2] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1125–1134. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2017/html/Isola_Image-To-Image_Translation_With_CVPR_2017_paper.html

[3] Center for Machine Perception, Czech Technical University in Prague, "CMP Facade Database." [Online]. Available: https://cmp.felk.cvut.cz/~tylecr1/facade/

[4] PyTorch Contributors, "PyTorch documentation." [Online]. Available: https://pytorch.org/docs/stable/index.html